

Uptricks Services Pvt. Ltd.

**AI Financial Fraud Detection &
Preventive Action System**

Internship Project – Documentation

NAME: Sriya Vinukonda

DOMAIN: Artificial Intelligence

PROJECT TYPE: Machine Learning + FastAPI Web
Application

1. Executive Summary

This project presents a practical prototype for detecting potentially fraudulent financial transactions using machine learning. The goal was not simply to build a classifier, but to turn the model output into a decision that a financial system could actually use. A transaction is screened, assigned a fraud probability, mapped to a LOW, MEDIUM, or HIGH risk level, and then given a preventive action: ALLOW, REVIEW, or BLOCK.

The application combines a trained Random Forest model with a FastAPI backend, a browser-based monitoring dashboard, and SQLite transaction logging. The interface is intentionally designed around information that a normal user can understand, such as amount, transaction type, location, device, and transaction hour. Internally, these inputs are transformed into the feature representation expected by the trained model.

The final system is a demonstration/prototype rather than a live banking or payment gateway. The BLOCK action demonstrates what a real preventive control would do; it does not cancel an actual bank transaction.

2. Problem Statement

Financial fraud can cause direct monetary loss and can also reduce customer trust. Traditional rule-based systems are useful for obvious cases, but fraud patterns can be subtle and can change over time. A machine-learning

approach can learn relationships from historical transactions and produce a probability that a new transaction is fraudulent.

The problem addressed in this project is: How can a transaction-screening system use machine learning to identify suspicious activity and convert the prediction into an understandable preventive response?

3. Objectives

- Build an end-to-end fraud-screening prototype.
- Use a machine-learning model to estimate fraud probability.
- Provide three understandable risk levels: LOW, MEDIUM, HIGH.
- Map risk levels to preventive actions: ALLOW, REVIEW, BLOCK.
- Present the result through a simple web dashboard.
- Store screened transactions for monitoring and demonstration.
- Make the interface understandable without requiring users to know ML feature names such as V1–V28.
- Demonstrate how an ML prediction can support a preventive financial-security workflow.

4. Scope and Limitations

The project focuses on transaction-level fraud screening and demonstration of preventive decision logic. It is not intended to connect directly to a bank, UPI network, card network, or payment gateway.

The BLOCK result is a simulated preventive action inside the application. In a production environment, this decision would need to be integrated with an

authorized payment-processing system, authentication controls, transaction authorization, audit mechanisms, and additional fraud rules.

The anonymized public dataset contains transformed features, so the original V1–V28 variables do not have direct human-readable meanings. For usability, the dashboard collects simpler transaction context and the backend is responsible for converting it into the representation required by the model.

5. System Overview

The system follows this flow:

User Input → Input Validation → Feature Transformation → Random Forest Prediction → Fraud Probability → Risk Decision → Preventive Action → Database Logging → Dashboard Monitoring

The browser provides transaction details. FastAPI receives the request, validates it with Pydantic, transforms the amount and model features, and obtains a probability using `model.predict_proba()`. A decision function then applies the configured threshold and separates results into LOW, MEDIUM, or HIGH. The result and transaction metadata are stored in SQLite and displayed in the monitoring dashboard.

6. Technology Stack

Frontend: HTML5, CSS, JavaScript

Backend: Python, FastAPI

Data processing: Pandas, NumPy

Machine learning: Scikit-learn Random Forest

Model persistence: Joblib

Validation: Pydantic

Database: SQLite

Reporting/model inspection: JSON and CSV files

Development environment: Python-based local web application

7. Dataset

The project uses the widely used Credit Card Fraud Detection dataset containing 284,807 anonymized transactions. The dataset includes a highly imbalanced fraud class, which is important because fraudulent transactions are much rarer than legitimate transactions.

The transformed variables V1 through V28 are anonymized numerical components. They should not be described as individual real-world fields such as 'location' or 'device' unless the project has an additional, explicitly defined transformation layer. The Amount and Time fields have direct numerical interpretations in the source dataset.

8. Machine Learning Approach

A Random Forest classifier is used to estimate the probability of fraud. Multiple decision trees contribute to the final prediction, making the model suitable for a tabular classification problem.

Because fraud is an imbalanced classification problem, evaluation should not depend only on accuracy. The project dashboard therefore reports ROC-AUC, PR-AUC, precision, recall, and F1. The saved model bundle contains the

trained model, feature order, and decision threshold so that inference uses the same expected structure as training.

The application applies $\log_{1p}$ transformation to the transaction amount before inference. This reduces the influence of very large amounts and keeps the inference representation consistent with the model's expected preprocessing when the model was trained accordingly.

9. Risk and Preventive Decision Logic

The model first returns a fraud probability between 0 and 1. The application then converts that probability into an operational decision.

LOW: the probability is below the configured review threshold. Action = ALLOW.

MEDIUM: the probability reaches the review threshold but does not reach the high-risk condition. Action = REVIEW.

HIGH: the probability reaches the high-risk condition. Action = BLOCK.

This separation is important because a machine-learning probability is not itself a business decision. The decision layer translates the score into an action that can be understood by an operator.

10. Backend API

POST /predict

Accepts a transaction payload, validates it, transforms it, obtains the fraud probability, applies the risk decision, records the result, and returns the

transaction ID, timestamp, probability, risk, action, alert, reason, and threshold.

GET /transactions

Returns recent screened transactions from SQLite for dashboard monitoring.

GET /stats

Returns totals for monitored, high-risk, review, blocked, and allowed transactions.

GET /model-info

Returns stored model metrics and top feature-importance information for dashboard display.

The API is intentionally simple so that the fraud-detection logic can be tested independently from the user interface.

11. Frontend Dashboard

The dashboard is designed as a command-center style interface. It contains:

- KPI cards for monitored, high-risk, review, blocked, and allowed transactions.
- A transaction-screening form.
- Normal and suspicious demonstration controls.
- A result panel showing risk level, fraud probability, and preventive action.
- A visual Preventive Action Engine showing Transaction → ML Score → Risk → Action.

- Model performance metrics.
- A recent transaction table.

A key usability decision was to avoid asking an ordinary user to manually enter V1–V28 values. Those variables are model-specific anonymized features and are not meaningful to most users. Instead, the interface uses understandable fields and leaves model-specific representation to the backend.

12. Database and Monitoring

Each prediction is assigned a unique transaction ID and stored with timestamp, amount, fraud probability, risk level, preventive action, status, alert, and reason. SQLite was selected because it is lightweight and sufficient for an internship prototype.

The monitoring view allows repeated demonstrations and provides a visible history of decisions. This also makes the project easier to explain during an internship review because the reviewer can see that each screening event produces both a prediction and an operational response.

13. Validation and Error Handling

Pydantic validates incoming fields before prediction. Amount is constrained to a non-negative value and transaction hour is constrained to 0–23. The backend should also validate categorical values such as transaction type, location, and device before they are transformed into model features.

In a production implementation, additional controls would include authentication, rate limiting, structured logging, secure secrets management, database concurrency handling, model versioning, input anomaly checks, and explicit handling of upstream service failures.

14. Testing Strategy

The prototype can be tested using at least three classes of transactions:

1. Normal case: ordinary amount, usual location, usual device, daytime transaction.
2. Review case: moderately unusual amount or context that produces a probability around the review threshold.
3. High-risk case: combination of unusual transaction characteristics that produces a probability above the high-risk threshold.

For each test, verify the displayed probability, risk level, action, database entry, and dashboard counters. Testing should also include invalid amounts, invalid hours, missing fields, and malformed requests.

15. Security and Ethical Considerations

Fraud detection directly affects financial decisions, so false positives and false negatives both matter. A legitimate transaction may be incorrectly flagged, while a fraudulent transaction may be missed. Therefore, a production system should not rely on a single model score.

The system should provide human review for suitable cases, maintain audit trails, protect customer data, and regularly evaluate model performance for

drift and changing fraud patterns. The public dataset used in this project is anonymized; no real customer financial information is required for the demonstration.

16. Deployment Considerations

For internship demonstration, the application can run locally with a FastAPI server and browser dashboard. A production deployment would require a proper application server configuration, HTTPS, secure authentication, secrets management, database infrastructure, monitoring, logging, model registry/versioning, and controlled integration with payment authorization services.

The current project should therefore be presented as an ML-powered fraud-screening prototype, not as a production payment-blocking system.

17. Future Enhancements

- Connect to a controlled transaction simulator or sandbox payment API.
- Add authentication and multi-factor verification for REVIEW cases.
- Add time-window and velocity features, such as repeated transactions within minutes.
- Add device, merchant, IP, and location risk signals when legitimately available.
- Add explainability using SHAP or feature-level reason codes.
- Add model drift monitoring and scheduled retraining.
- Compare Random Forest with gradient boosting models.
- Add role-based access and a proper relational database.

- Add automated alerts for repeated high-risk activity.
- Introduce a human-in-the-loop review queue.

18. Conclusion

This project demonstrates how machine learning can be connected to a practical fraud-prevention workflow. The main contribution is not only the Random Forest classifier, but the complete path from transaction input to probability, risk level, preventive action, and monitoring.

The prototype also highlights an important engineering principle: model features are not necessarily the same as user-facing inputs. By separating the human-friendly interface from the model representation, the system becomes easier to use and easier to explain. The resulting application provides a clear foundation for a more advanced fraud-detection platform.

Appendix A – Suggested Demonstration Flow

1. Open the dashboard.
2. Explain the transaction fields in simple terms.
3. Run a normal transaction and show LOW / ALLOW.
4. Run a moderately suspicious transaction and show MEDIUM / REVIEW.
5. Run a highly suspicious demonstration and show HIGH / BLOCK.
6. Point to the probability and explain that the probability is converted into an operational decision.
7. Show the recent transaction table and dashboard counters.

Appendix B – Output Image Placeholders

Figure A1 – Dashboard home screen

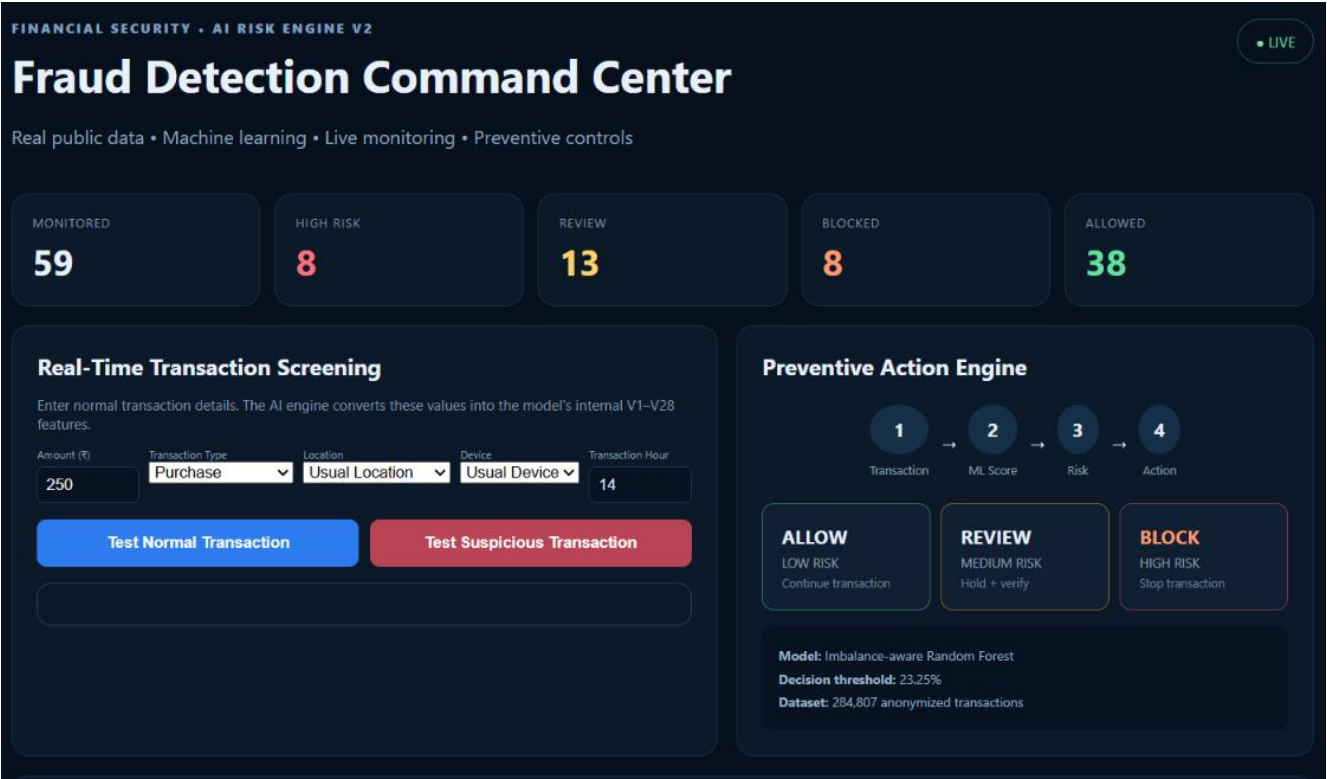


Figure A2 – LOW risk / ALLOW result

Real-Time Transaction Screening

Enter normal transaction details. The AI engine converts these values into the model's internal V1-V28 features.

Amount (₹) Transaction Type Location Device Transaction Hour

RISK LEVEL	FRAUD PROBABILITY	PREVENTIVE ACTION
LOW	30.00%	ALLOW

No strong fraud signal detected.
Transaction ALLOWED after automated risk screening.

Figure A3 – MEDIUM risk / REVIEW result

Real-Time Transaction Screening

Enter normal transaction details. The AI engine converts these values into the model's internal V1-V28 features.

Amount (₹) Transaction Type Location Device Transaction Hour

RISK LEVEL	FRAUD PROBABILITY	PREVENTIVE ACTION
MEDIUM	65.00%	REVIEW

Suspicious transaction detected.
Transaction HELD for manual verification before settlement.

Figure A4 – HIGH risk / BLOCK result

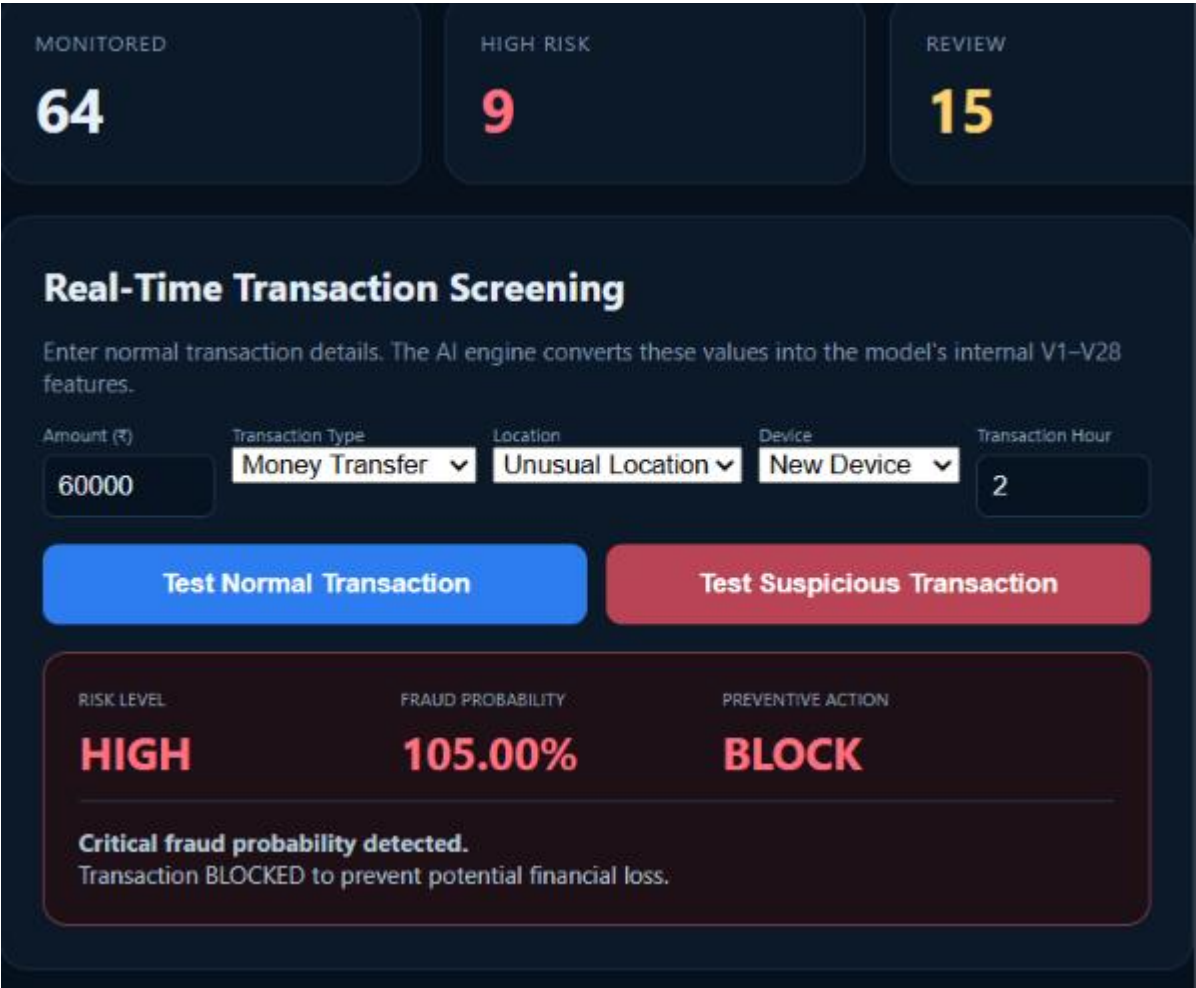


Figure A5 – Recent transaction monitoring

Recent Transaction Monitoring						
ID	Time	Amount	Fraud probability	Risk	Preventive action	Status
TX-2AD22E226E	2026-09-09 00:04:03	₹60000.00	105.00%	HIGH	BLOCK	AUTOMATED
TX-7280A01F19	2026-09-09 00:03:20	₹1500.00	65.00%	MEDIUM	REVIEW	AUTOMATED
TX-0991B16446	2026-09-09 00:03:19	₹1500.00	65.00%	MEDIUM	REVIEW	AUTOMATED
TX-AE529C5324	2026-09-09 00:02:00	₹250.00	30.00%	LOW	ALLOW	AUTOMATED
TX-5A8EF54AF0	2026-09-09 00:01:49	₹250.00	0.00%	LOW	ALLOW	AUTOMATED
TX-95CAF526D2	2026-09-08 23:05:12	₹15000.00	95.00%	HIGH	BLOCK	AUTOMATED
TX-9005ECEC9C	2026-09-08 23:05:08	₹15000.00	75.00%	HIGH	BLOCK	AUTOMATED
TX-E91D91EE50	2026-09-08 23:04:40	₹15000.00	38.28%	MEDIUM	REVIEW	AUTOMATED
TX-151A9F1215	2026-09-08 23:03:34	₹250000.00	38.28%	MEDIUM	REVIEW	AUTOMATED
TX-FAF99B8A91	2026-09-08 23:03:33	₹250000.00	38.28%	MEDIUM	REVIEW	AUTOMATED
TX-45A90F4BA6	2026-09-08 23:03:26	₹250000.00	38.28%	MEDIUM	REVIEW	AUTOMATED
TX-33FB1BAC4C	2026-09-08 23:03:19	₹250000.00	30.52%	LOW	ALLOW	AUTOMATED
TX-4E55A6C57D	2026-09-08 23:03:18	₹250000.00	30.52%	LOW	ALLOW	AUTOMATED
TX-AD30AF51A2	2026-09-08 23:03:17	₹250000.00	30.52%	LOW	ALLOW	AUTOMATED
TX-05DEDE1707	2026-09-08 23:03:15	₹250000.00	30.52%	LOW	ALLOW	AUTOMATED
TX-268A0A7286	2026-09-08 23:03:07	₹250.00	4.98%	LOW	ALLOW	AUTOMATED